Empirical Analysis of Insertion Sort

Objective

Implement insertion sort, count key comparisons and shifts, measure performance on sorted, reverse-sorted, and random arrays, and compare with theoretical growth and merge sort.

Methodology

Python 3; n = 100, 250, 500, 1,000, 2,000, 3,000, 4,000, 5,000. Each time is the median of three measurements using time.perf_counter(). Random data uses a fixed seed. Every output was checked against Python's sorted result. A shift is an assignment moving an existing element one position right.

Results at n = 5,000

Input arrangement	Comparisons	Shifts	Median time (s)
Sorted	4,999	0	0.000513
Reverse-sorted	12,497,500	12,497,500	1.432642
Random	6,116,226	6,111,237	0.749601

Analysis

Sorted input is the best case: n–1 comparisons and no shifts, consistent with O(n). Reverse-sorted input requires n(n–1)/2 comparisons and shifts, consistent with $O(n^2)$; random input also showed quadratic-like work. For random n = 5,000, merge sort took 0.008102 s versus 0.749601 s for insertion sort in this run. Merge sort has O(n log n) time in all cases but uses extra memory; insertion sort is in-place and effective for small or nearly sorted arrays.

Conclusion

Measurements support the theoretical analysis: insertion sort performs well on sorted or small inputs but scales poorly on reverse-sorted and random large inputs. Merge sort was substantially faster on the 5,000-element random input in this environment. Wall-clock times vary by machine and system load.

References

1. Cormen, Leiserson, Rivest, and Stein, *Introduction to Algorithms*, 4th ed., MIT Press, 2022. 2. Python documentation, time.perf_counter(): https://docs.python.org/3/library/time.html#time.perf_counter